Meridian

Secure Job Search & Professional Networking Platform

Milestone 1 Report

Tech Stack Finalization, HTTPS Setup, Skeleton Deployment

CSE 345/545 — Foundations of Computer Security

Team Members	Roll Number
Aniket Gupta	2022073
Angadjeet Singh	2022071
Harsh Kumar	2022198

IIIT Delhi

February 13, 2026

Contents

1 Overview

This report covers Milestone 1 of the Meridian platform, due February 13, 2026 (no credit). The milestone required three deliverables:

1. Finalize the technology stack
2. Configure HTTPS with TLS certificates
3. Deploy a skeleton application on the VM

All three goals are complete. This report documents the decisions made and the rationale behind each choice.

2 Technology Stack

Each technology was selected based on its security properties and development ergonomics.

2.1 Backend — Python / FastAPI

We chose FastAPI over Django and Express for three reasons:

- **Dependency injection** — FastAPI's `Depends()` system cleanly separates authentication and RBAC from route logic. Every protected endpoint declares its security requirements as function parameters, making access control auditable at a glance.
- **Pydantic schemas** — All incoming data passes through typed schemas with validators. Malformed input is rejected before reaching business logic or the database.
- **Async-native** — Built on ASGI (Uvicorn), enabling non-blocking I/O for concurrent request handling without thread-safety concerns.

2.2 Frontend — React 18 / Vite

React was chosen for its component model and built-in XSS protection — all JSX expressions are automatically escaped. Vite provides fast hot module replacement during development and optimized production builds.

2.3 Database — PostgreSQL 16

PostgreSQL offers features critical to our security requirements:

- **Native UUID columns** — Used as primary keys to prevent ID enumeration attacks
- **JSONB** — Flexible storage for audit log details without schema changes per event type
- **ACID compliance** — Transaction integrity for sensitive operations like account creation and session management
- **Parameterized queries** — Via SQLAlchemy ORM, structurally preventing SQL injection

2.4 Summary

Layer	Technology	Security Rationale
Backend	Python 3.11 / FastAPI	DI-based auth, Pydantic validation
Frontend	React 18 / Vite	JSX auto-escaping (XSS defense)
Database	PostgreSQL 16	UUID PKs, JSONB, parameterized queries
Web Server	Nginx	TLS termination, reverse proxy, headers
Deployment	Docker + Compose	Reproducible, isolated environments
CI/CD	GitHub Actions	Automated lint, test, build

Table 1: Technology stack with security rationale.

3 HTTPS Configuration

All traffic is served over HTTPS with the following setup:

3.1 Certificate Setup

For the development environment on the course VM, we generated self-signed certificates using OpenSSL:

```
openssl req -x509 -nodes -days 365 \
    -newkey rsa:2048 \
    -keyout ssl/private.key \
    -out ssl/certificate.crt \
    -subj "/CN=meridian.local"
```

3.2 Nginx TLS Configuration

Nginx is configured as a reverse proxy with TLS termination:

- Listens on port 443 (HTTPS) with the self-signed certificate
- Port 80 (HTTP) redirects all traffic to HTTPS
- `Strict-Transport-Security` header enabled with 1-year max-age
- Proxies `/api/*` to the FastAPI backend (port 8000)
- Serves the React frontend build as static files

3.3 Security Headers

Even at this early stage, we configured security headers on all responses:

```
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-XSS-Protection: 1; mode=block
Strict-Transport-Security: max-age=31536000; includeSubDomains
Referrer-Policy: strict-origin-when-cross-origin
```

4 Skeleton Application

The skeleton deployment proves that all components work together end-to-end.

4.1 What Was Deployed

- **FastAPI backend** with health check endpoint (`GET /api/health`)
- **React frontend** with landing page, routing, and navigation shell
- **PostgreSQL database** with initial schema (users table)
- **Docker Compose** orchestrating all three services
- **Environment config** via `.env.example` documenting all required variables

4.2 Project Structure

```
Secure-Job-Portal/
  backend/
    app/
      main.py            # FastAPI app with middleware
      config.py           # Environment-based settings
      database.py         # SQLAlchemy engine + session
    Dockerfile
    requirements.txt
  frontend/
    src/
      pages/             # React page components
      components/        # Reusable UI components
      context/           # Auth and theme context
    Dockerfile
    nginx.conf
  docker-compose.yml
  .env.example
```

4.3 Verification

The skeleton was verified by:

1. Accessing the frontend via HTTPS (self-signed cert warning expected)
2. Hitting the `/api/health` endpoint and confirming a 200 response
3. Confirming PostgreSQL connectivity via Docker Compose logs
4. Verifying all security headers are present on responses

5 What's Next — Milestone 2

Milestone 2 (February 27, 2.5%) will build on this foundation with:

- Secure user registration and login (Argon2id, JWT, account lockout)
- TOTP-based two-factor authentication
- User profile management with input sanitization
- Secure resume upload with Fernet encryption at rest
- Basic admin dashboard with RBAC and audit logging